RasberryPi OS 64bit + Docker + PiHole (Primary) + Unbound 9s DoT + KeepAliveD (Part 1)

Saturday, January 3, 2026 3:53 PM

RasberryPi OS 64bit + Docker + PiHole (Primary) + Unbound 9s DoT + KeepAliveD (Part 1)

Created: 2026-10-10

Objective:

1. Create a Primary PiHole container that becomes a DNS server with network wide sinkhole to block ads, tracking, and add a bit more security.

2. Configure Unbound DNS w/ DoT.

3. Leverage KeepAliveD to create an HA pair of DNS servers for less down time.

Prerequisites/Notes:

1. This guide is one of two as this will be the primary PiHole server and the other will be syncing all changes from the primary. These two Pis will also serve as a HA pair with KeepAliveD. I configured these two server via a windows box.

2. In this guide, you will make your Pi own the following parts in DNS,

   Forward Resolver: (Unbound w/ 9.9.9.9 + DoT)

   Forwards requests past ISP DNS upstream servers and goes right to 9s upstream encrypted with DoT. The IPS can only see you used it but not traffic. Also 9s can see your traffic but states it does not keep identifiable user records them over.

   A Caching Server: (Unbound)

   The Pi remembers every successful DNS request it makes. The next time a device on your network asks for the same website, the Pi pulls the answer straight from its RAM instantly, saving bandwidth and reducing latency.

   A Forwarding Sinkhole: (PiHole)

   Before any internet searching happens, the Pi acts as a selective forwarder. It checks incoming requests against your blocklists. If a request is for an ad tracker or malware domain, it "sinkholes" it (blocks it); if it is safe, it forwards it down to Unbound.

   Keepalived uses VRRP to float one virtual IP between your two Pi-holes, so if the active one dies, the backup grabs the IP within seconds and clients never notice.

3. Hardware list:

   - Two Raspberry Pi 3B.
   - Two high endurance SD cards.
   - An SD card reader of some sort is needed to flash/install the RasberryPi OS to the microSD card.
   - Two ethernet cables
   - Switch
   - Router

4. In this guide the Pi's are connected via ethernet and that switch is directly connected to the primary router. You can use Wi-Fi for a PoC but this is not ideal due to latency and band speed reasons. Thought the 3B supports 802.11ac, the max throughput is only 200–250 Mbps. So even though this pi model has a 1GB Nic, its max throughput is 300–330 Mbps.

5. Though multiple pies will be leveraged in these guides, the VRRP IP must be specified as the only DNS server either by your Router/Firewall or devices will bypass a PiHole.

Steps:

Installing the OS to Storage:

Install Raspberry Pi OS 64 Bit on your 3B pi. I used the Raspberry Pi image and an high endurance microSD card. Some cards might come write protected or the micro adapter might be write locked. To unwrite protect a card on Windows, use Diskpart.

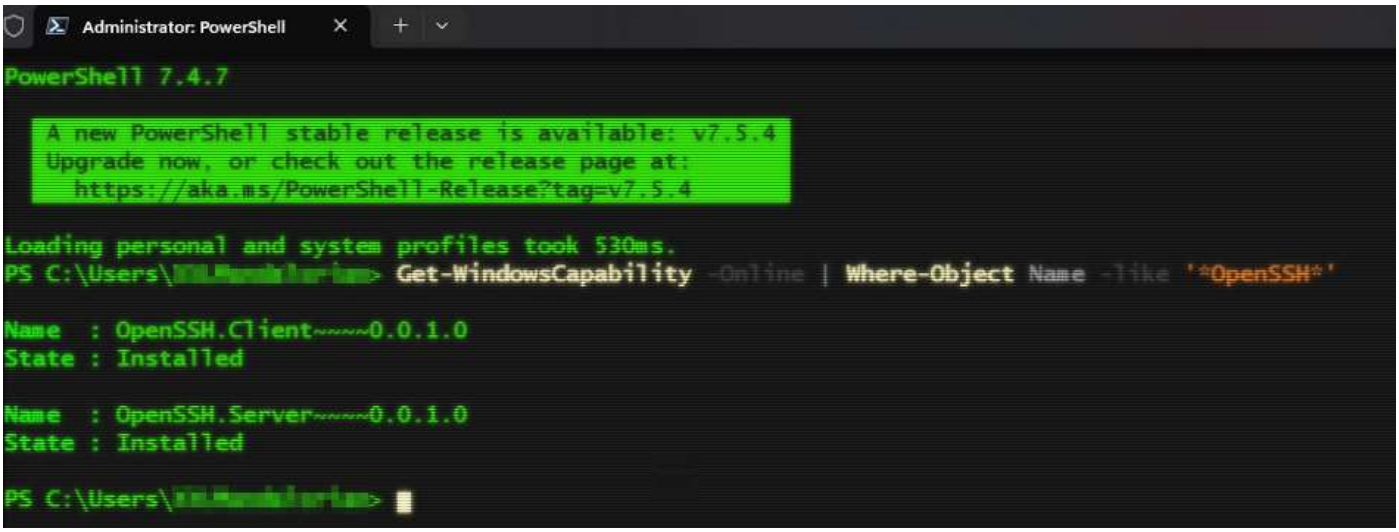
1. Type Diskpart and press Enter.
2. Type list disk and find the disk number of your micro SD card (check by size).
3. Type select disk X (replace X with your card's number).
4. Type attributes disk clear readonly and press Enter.

Disable the GUI (Optional):

Disable the UI boot to save some compute.

5. Ensure openssh is installed on your windows box. Ensure both the client and server are installed.

   Get-WindowsCapability -Online | Where-Object Name -like '*OpenSSH*'

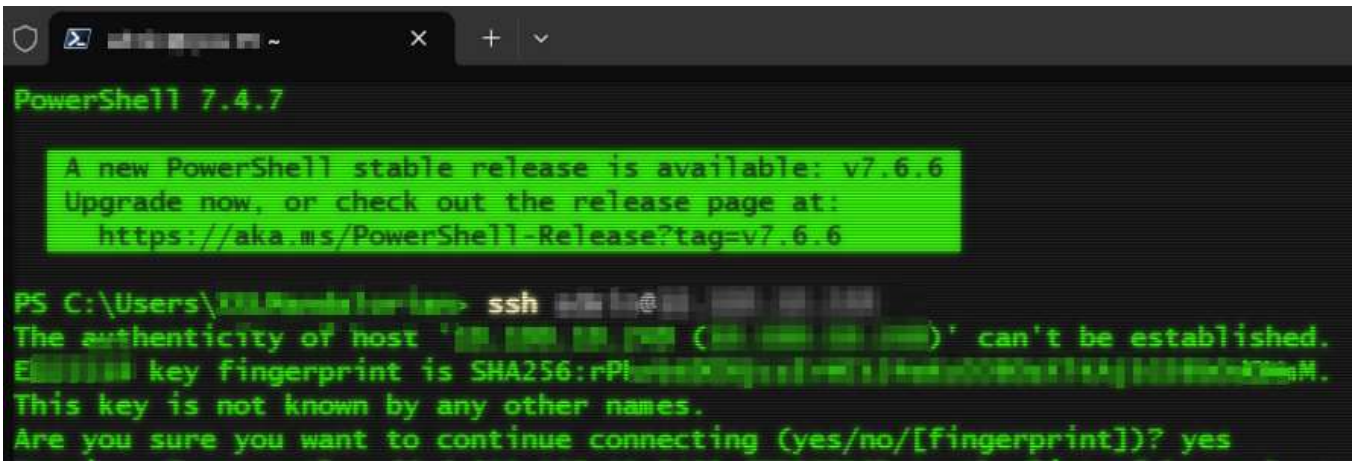


   If it's not installed run the following, no reboot needed.

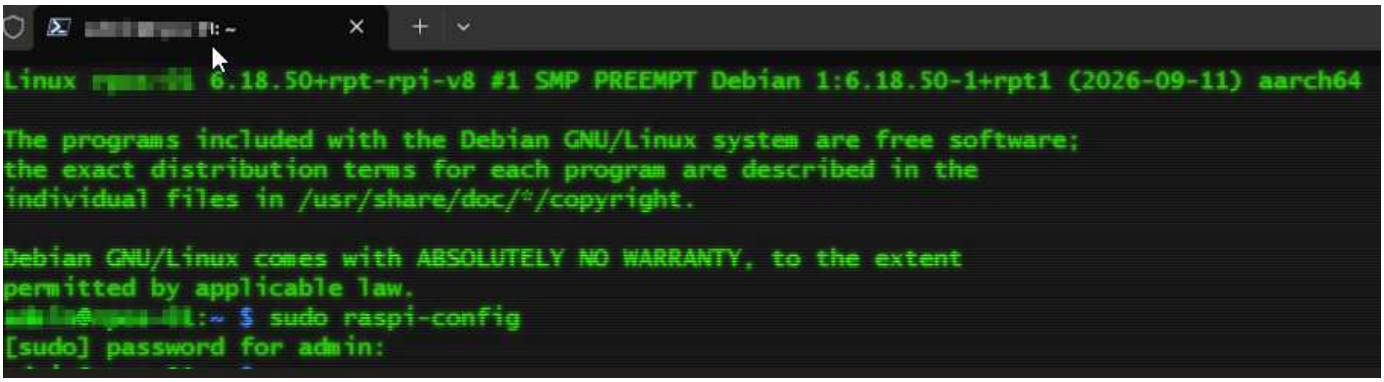
   Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0

6. SSHing into the device, Yes if you trust the device.

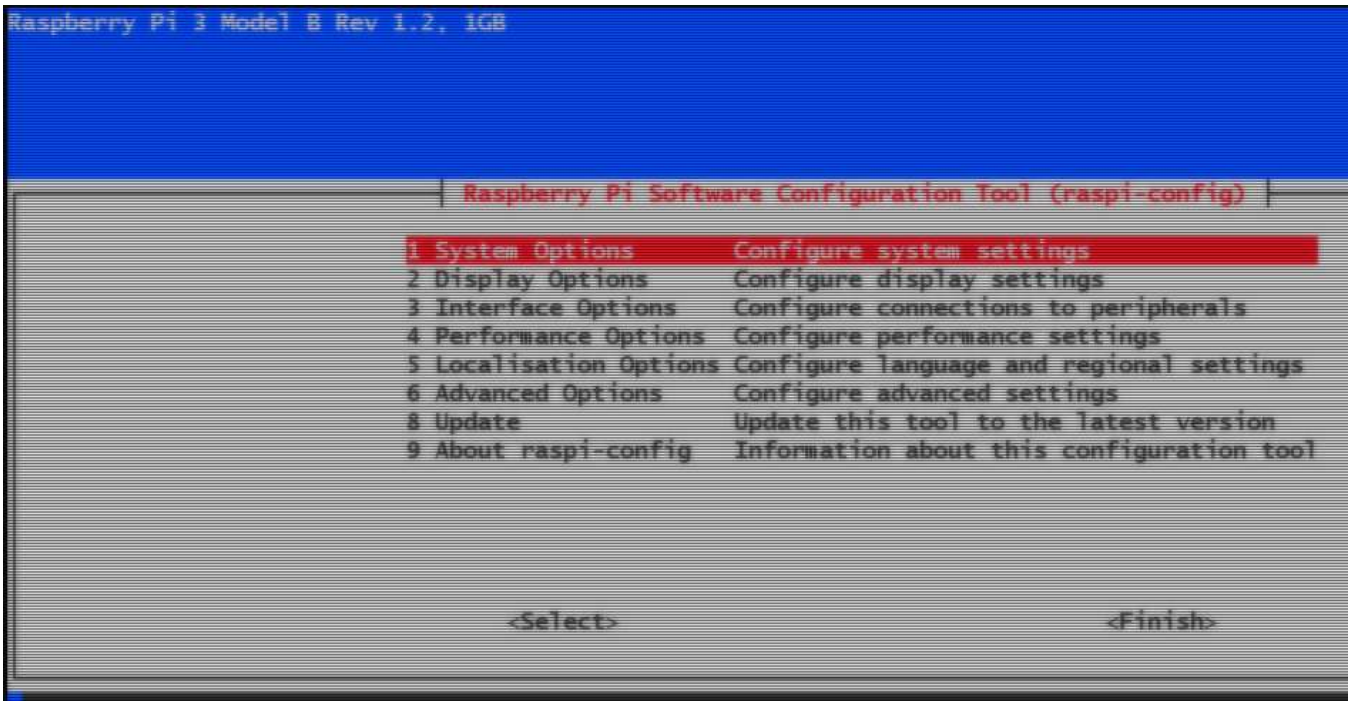
   ssh UserNameHere@YourIPHere

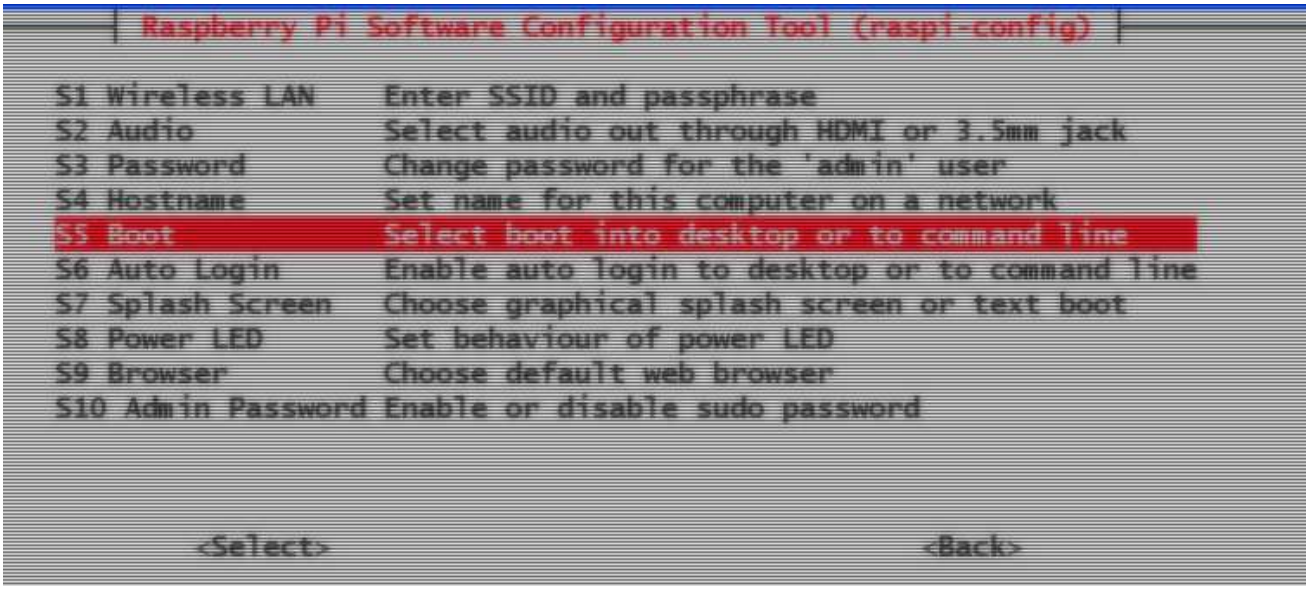


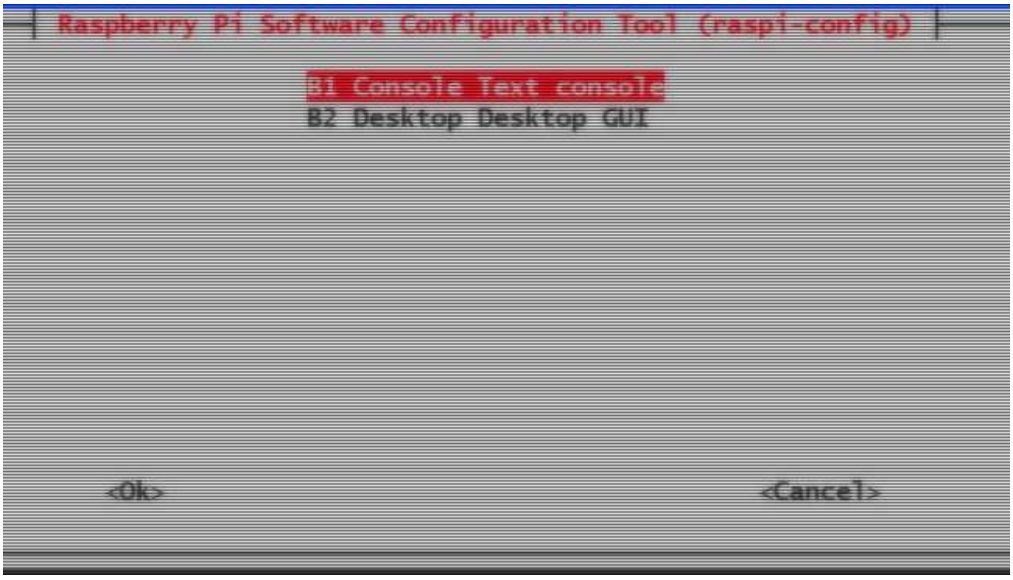
7. sudo raspi-config

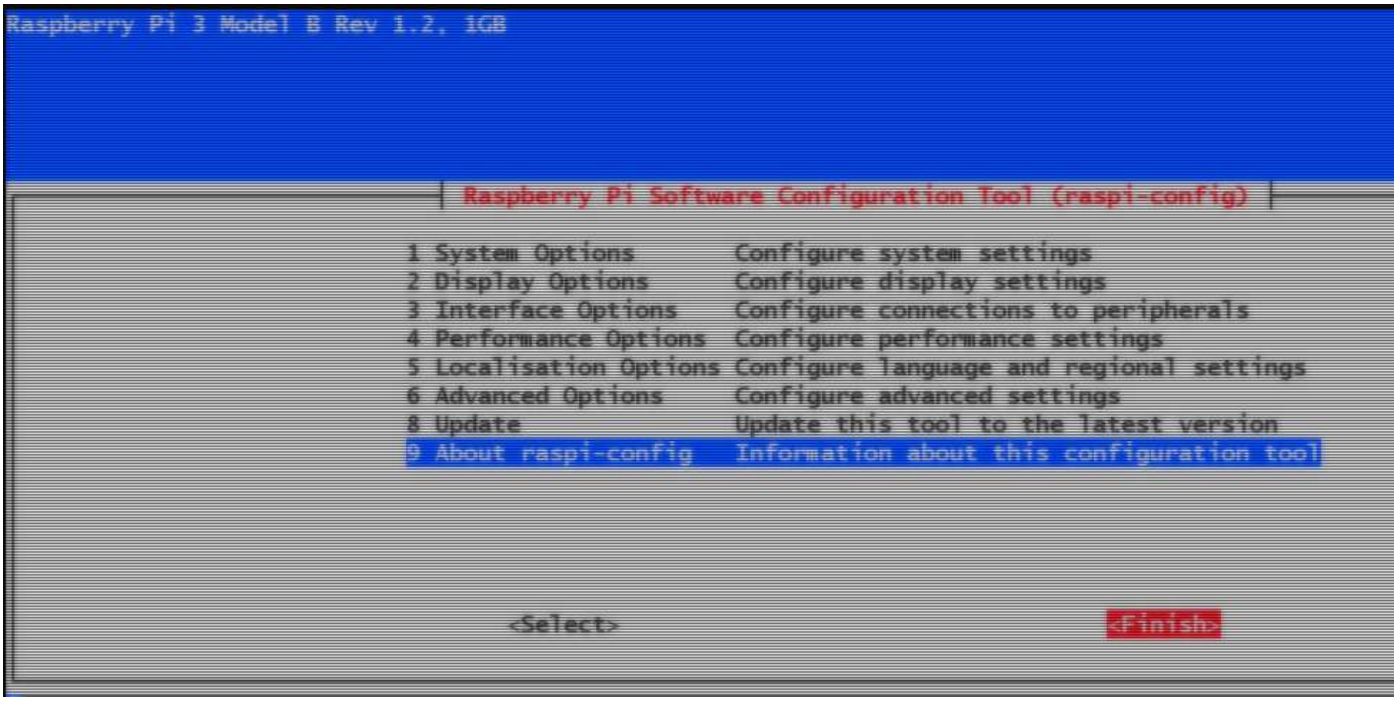


4. Select System Options > S5 Boot > B1 Console Text console > Enter. Note, it might shoot you back to the SHH cli session for a moment. Wait for it to come back to this TUI > Finish.

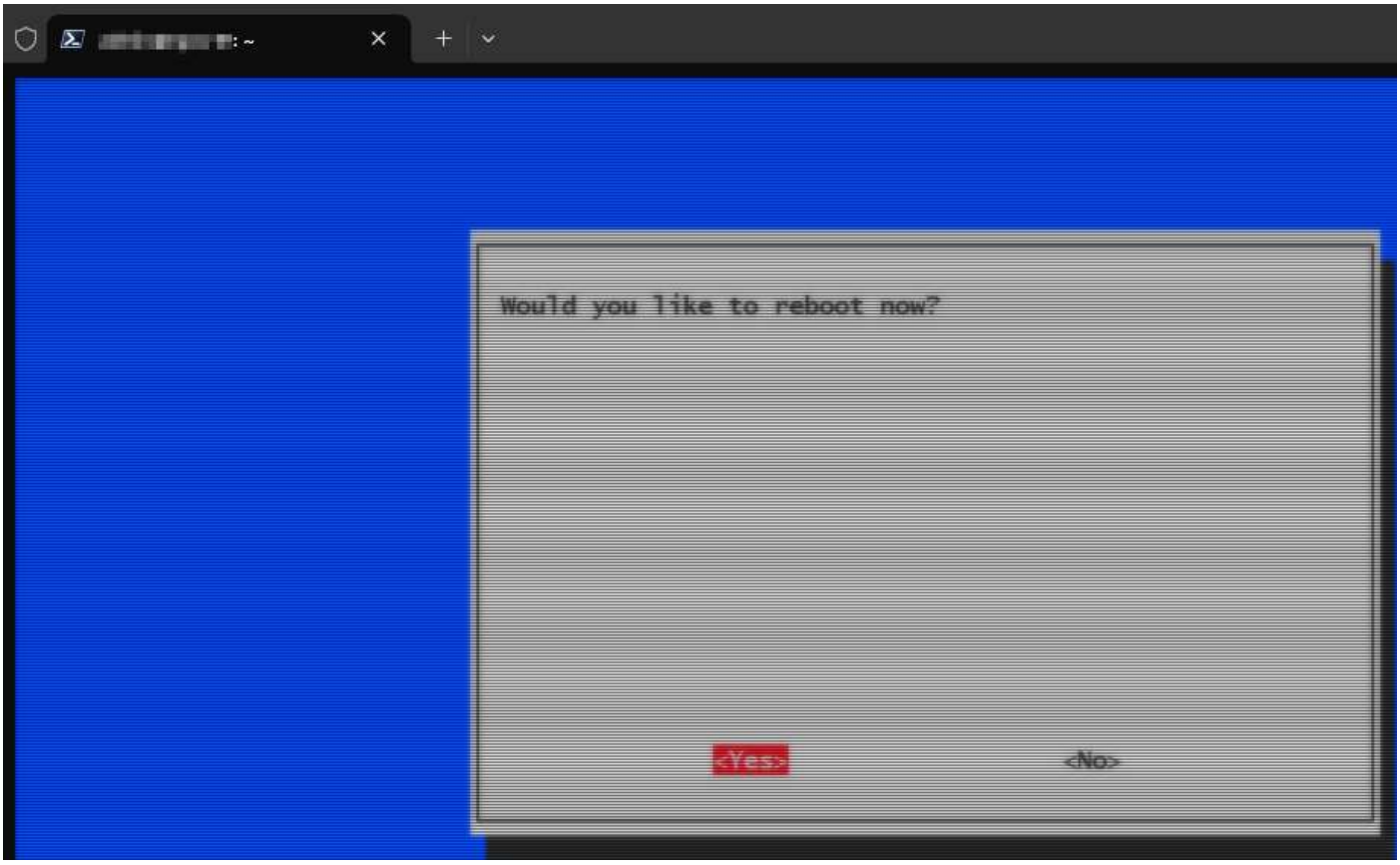








5. Reboot prompt, yes.

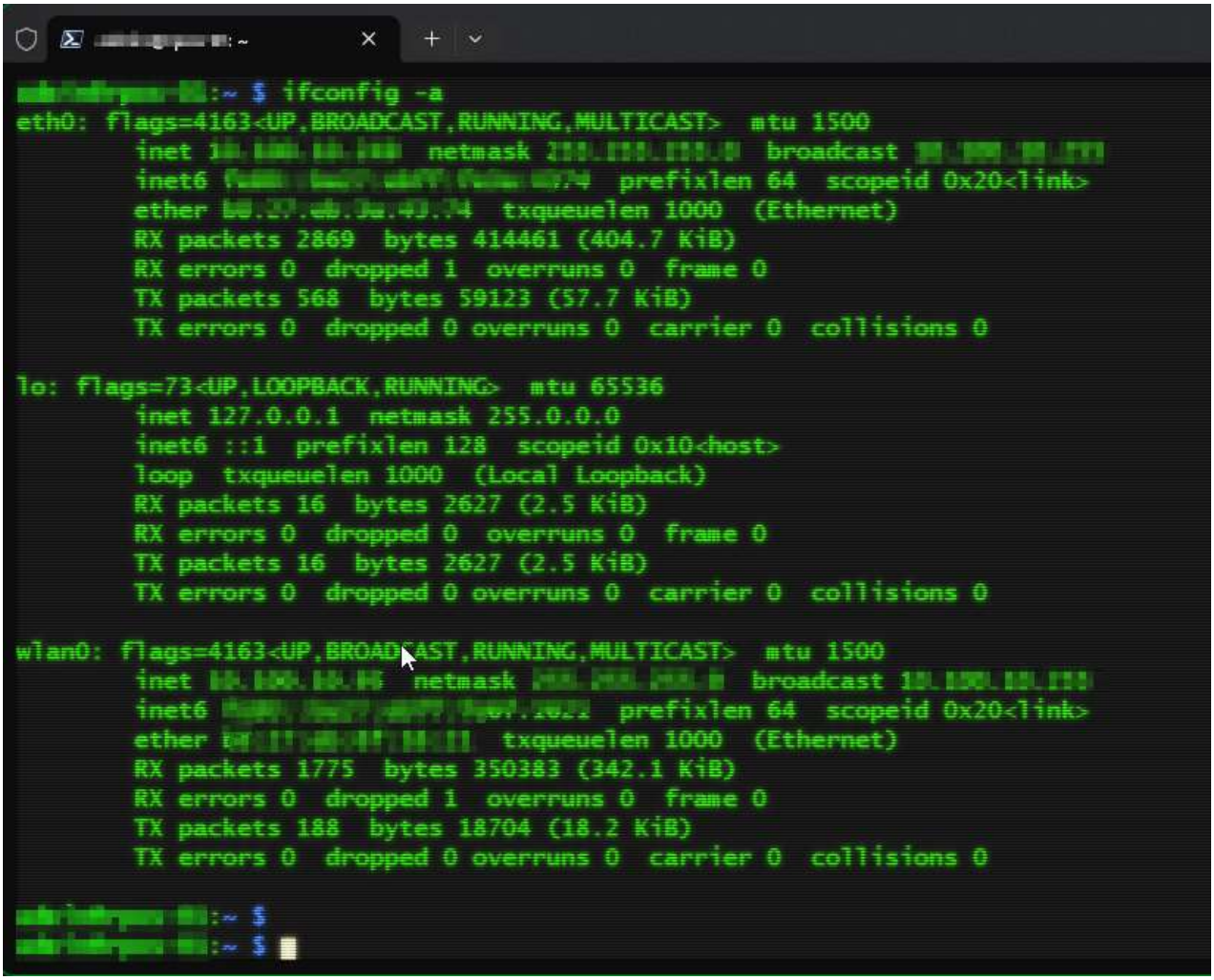




Statically set the IP:

1. Find your ethernet link that is up. You can use wifi but this is really not recommended.

   ifconfig -a



2. As this OS supports Network Manager you can use my shell script to statically set the IPV4 address on a link. Modify it to your needs in a text editor. This can be for eth or wifi, just ensure you specify the link name as so. Note, it will disable IPV6 unless modified. Be sure to check/modify your DHCP reservations before and after making changes. It's a good idea to pick out the secondary Pi's IP at this time too as it will be needed in the KeepAliveD portion of this guide.

   Note, point the DNS to whatever you plan to use going forward. I will used 9.9.9.9 and 1.1.1.2. I would recommend putting in two w/ different providers, should one be down. The Pis should never be pointed to the final virtual IP or the Pis directly, or the router when the VIP is in place on the router for DNS as they will not resolve after reboot/shutdown. If you are doing this configuration remote, it might be ideal to have Wi-Fi connected to ensure you set the static eth right or the inverse, then disable the unused later.

   https://github.com/XXLMandalorian013/Linux/blob/main/Scripts/Network/nmcli/ethernet/connect_ethernet_static.sh

3. Now that you have modified the script to your needs let's make a .sh file to run on the Pi if you have not already. If you save the script to /tmp it will be deleted after a reboot.

   nano /home/admin/Documents/connect_ethernet_static.sh



   Paste in your changes and save/exit. Note ^ = Ctl on windows.

```
GNU nano 8.4                                    /home/[redacted]/Documents/co
#!/bin/bash
# vars
conn_name="ethernet-static"
iface="eth0"
method="manual"
ipv4="XXX.XXX.XXX.XXX/24"
gateway="XXX.XXX.XXX.XXX"
dns1="9.9.9.9"
dns2="1.1.1.2"
disable="disabled"
## script start ##
# check if the connection exists, if not, create it
if nmcli con show "$conn_name" >/dev/null 2>&1; then
    echo "$conn_name exists, continuing..."
else
    echo "$conn_name does not exist, creating connection..."
    sudo nmcli con add type ethernet \
        ifname "$iface" \
        con-name "$conn_name"
fi
# configure connection
sudo nmcli con mod "$conn_name" \
    connection.interface-name "$iface" \
    ipv4.method manual \
    ipv4.addresses "$ipv4" \
    ipv4.gateway "$gateway" \
    ipv4.dns "$dns1 $dns2" \
    ipv4.ignore-auto-dns yes \
    ipv6.method "$disable"
# enable the connection
sudo nmcli con up "$conn_name"
## script end ##
```

Be sure to cat the fine to ensure the vars look good.

```
[redacted]:~/Documents $ cat /home/[redacted]/Documents/connect_ethernet_static.sh
```

4. Now run the script. Note, you will lose your current SSH connection if the IP changed.

   chmod +x /home/YourNameHere/Documents/connect_ethernet_static.sh

   /home/YourNameHere/Documents/connect_ethernet_static.sh

```
[redacted]:~/Documents $ chmod +x /home/[redacted]/Documents/connect_ethernet_static.sh
[redacted]:~/Documents $ /home/[redacted]/Documents/connect_ethernet_static.sh
ethernet-static does not exist, creating connection...
Connection 'ethernet-static' (a48[redacted]7bb) successfully added.
client_loop: send disconnect: Connection reset
```

5. Now reboot. As we statically set the IP, be sure to SSH using the new static address to ensure its working.

   sudo reboot now

```
[redacted]:~ $ sudo reboot now
[redacted]:~ $ client_loop: send disconnect: Connection reset
```

SSH back into the device via the static setting to ensure its sticking.

6. If all is working on eth disable wifi from auto connecting to prevent issues.

   nmcli connection show

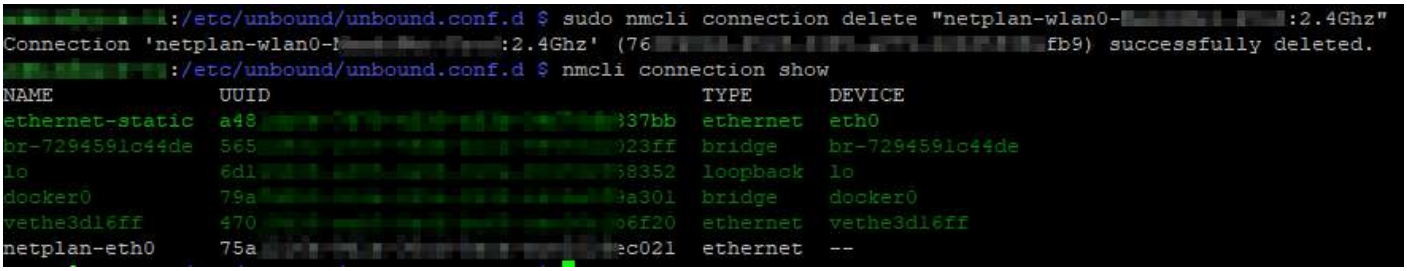


   Delete the wifi SSID so it can't auto connect. It can be re-added if something come up.

   sudo nmcli connection delete "SSIDNameHere"

   Then ensure its delete,

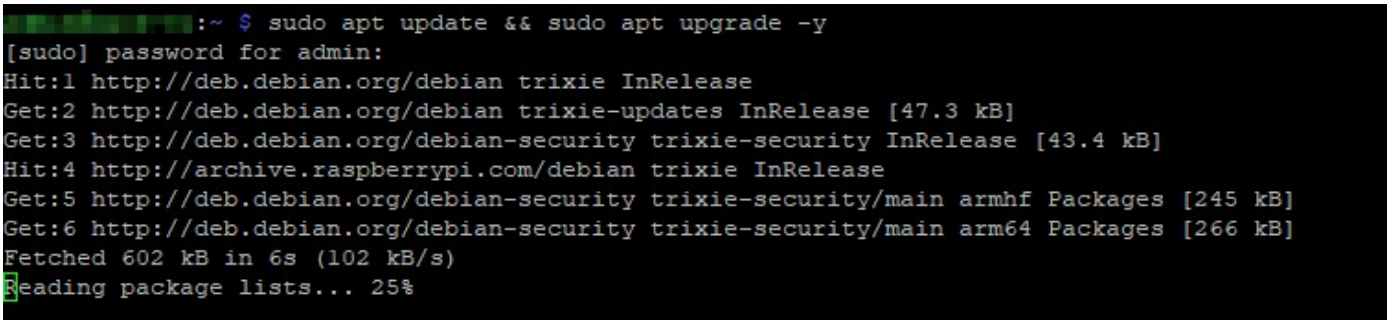
   nmcli connection show



   Update the OS and Packages:

7. Pulls the latest apt repo changes and updates the installed software packages.

   sudo apt update && sudo apt upgrade -y

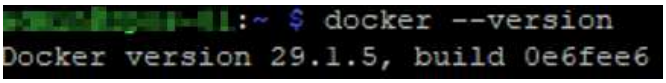


   Install and configure Docker:

   curl -fsSL https://get.docker.com -o get-docker.sh
   sudo sh get-docker.sh

   And ensure it installed.

   docker --version



   I also wrote this script you can run instead to install docker. Like before, copy it to the pi and run it. This one can be saved in /tmp as we only need to use it once.

   https://github.com/XXLMandalorian013/Linux/blob/main/Scripts/Software/Installers/docker/64_bit/install_docker_64bit.sh

nano /tmp/install_docker_64bit.sh

```
:/ $ nano /tmp/install_docker_64bit.sh
```

sudo /tmp/install_docker_64bit.sh

```
:/ $ sudo /tmp/install_docker_64bit.sh
docker is not installed...installing docker...
URL is reachable and returned 200 OK..downloading installer...
running docker installer...
# Executing docker install script, commit: 2b32480025b223ebfddae9a3a8bef09027680f53
+ sh -c apt-get -qq update >/dev/null
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get -y -qq install ca-certificates curl >/dev/null
+ sh -c install -m 0755 -d /etc/apt/keyrings
+ sh -c curl -fsSL "https://download.docker.com/linux/debian/gpg" -o /etc/apt/keyrings/docker.asc
+ sh -c chmod a+r /etc/apt/keyrings/docker.asc
```

```
The docker service is running...docker installed correctly...ending script...
:/ $
```

8. Ensure docker and future containers like pihole starts each restart/shutdown of the system.

   sudo systemctl start docker

   sudo systemctl enable docker

```
:/home $ sudo systemctl start docker
:/home $   sudo systemctl enable docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
```

Installing PiHole:

1. I like to create a dir in /opt to keep all my containers.

   sudo mkdir -p /opt/docker_containers

```
:/opt $ sudo mkdir -p /opt/docker_containers
:/opt $ ls
containerd  docker_containers  WidevineCdm
:/opt $
```

2. Now create a dir specifically for the pihole container .yaml file.

   sudo mkdir -p /opt/docker_containers/pi_hole

```
:/opt/docker_containers $ sudo mkdir -p /opt/docker_containers/pi_hole
:/opt/docker_containers $ ls
pi_hole
:/opt/docker_containers $
```

3. Now make a persistent volume location for pihole. If we don't make it and the compose file makes it, there could be access issues down the road.

   sudo mkdir -p /opt/docker_containers/pi_hole/volume

```
:/opt/docker_containers $ sudo mkdir -p /opt/docker_containers/pi_hole/volume
```

```
:/opt/docker_containers/pi_hole $ ls
volume
```

4. Now make the docker compose .yaml file.

   sudo touch /opt/docker_containers/pi_hole/pi_hole_conf.yaml

```
[redacted]:/opt/docker_containers/pi_hole $ sudo touch /opt/docker_containers/pi_hole/pi_hole_conf.yaml
[redacted]:/opt/docker_containers/pi_hole $ ls
pi_hole_conf.yaml  volume
[redacted]:/opt/docker_containers/pi_hole $
```

5. Then edit that .yaml.

   sudo nano /opt/docker_containers/pi_hole/pi_hole_conf.yaml

```
[redacted]/opt/docker_containers/pi_hole $ sudo nano /opt/docker_containers/pi_hole/pi_hole_conf.yaml
```

6. Now edit the .yaml.

```
[redacted]:/opt/docker_containers/pi_hole $ sudo nano /opt/docker_containers/pi_hole/pi_hole_conf.yaml
```

7. Edit and past in the following. Note, pay attention to the spacing, don't forget to change the password to what you want it to be, and be sure to save before exiting. Use the PiHole quick start template on their GitHub page to meet your needs. https://github.com/pi-hole/docker-pi-hole/#quick-start Below is how I configure my .yaml file.

   pi_hole_conf

```
[redacted]
 GNU nano 8.4                                                    /opt/docker_containers/pi_hole/pi_hole_conf.yaml *
# More info at https://github.com/pi-hole/docker-pi-hole/ and https://docs.pi-hole.net/
services:
  pihole:
    container_name: [redacted]
    image: pihole/pihole:latest
    restart: unless-stopped

    ports:
      # DNS Ports
      - "53:53/tcp"
      - "53:53/udp"
      # Default HTTP Port
      - "80:80/tcp"
      # Default HTTPS Port. FTL will generate a self-signed certificate
      - "443:443/tcp"
      # Uncomment if using Pi-hole as DHCP
      # - "67:67/udp"
      # Uncomment if using Pi-hole as NTP
      # - "123:123/udp"

    environment:
      # Timezone
      TZ: "America/New_York"
      # Web UI password
      FTLCONF_webserver_api_password: "[redacted]"
      # Required when using bridge networking
      FTLCONF_dns_listeningMode: "all"

    volumes:
      # Persist Pi-hole data dir. Be sure to add the following after your desired pihole container dir. etc-pihole:/etc/pihole, else the container will break.
      - "/opt/docker_containers/pi_hole/volume/etc-pihole:/etc/pihole"
      # Optional custom dnsmasq configs
      # - "./etc-dnsmasq.d:/etc/dnsmasq.d"

    # cap_add:
    #   - NET_ADMIN
```

8. It's a good idea to validate the .yaml file by double checking the formatting.

   cd /opt/docker_containers/pi_hole/

```
[redacted]:/ $ cd /opt/docker_containers/pi_hole
```

   docker compose -f pi_hole_conf.yaml config

```
:/opt/docker_containers/pi_hole $ docker compose -f pi_hole_conf.yaml config
name:
services:
  pihole:
    container_name:
    environment:
      FTLCONF_dns_listeningMode: all
      FTLCONF_webserver_api_password:
      TZ: America/Chicago
    image: pihole/pihole:latest
    networks:
      default: null
    ports:
      - mode: ingress
        target: 53
        published: "53"
        protocol: tcp
      - mode: ingress
        target: 53
        published: "53"
        protocol: udp
      - mode: ingress
        target: 80
        published: "80"
        protocol: tcp
      - mode: ingress
        target: 443
        published: "443"
        protocol: tcp
    restart: unless-stopped
    volumes:
      - type: bind
        source: /opt/docker_containers/pi_hole/etc-pihole
        target: /etc/pihole
        bind: {}
networks:
  default:
    name: pi_hole_default
:/opt/docker_containers/pi_hole $
```

9. Add a sudo/wheel user to the docker admin group. If we don't we will see,

```
:/opt/docker_containers/pi_hole $ docker compose -f pi_hole_conf.yaml up -d
unable to get image 'pihole/pihole:latest': permission denied while trying to connect to t
ihole:latest/json": dial unix /var/run/docker.sock: connect: permission denied
```

sudo usermod -aG docker YourUserNameHere

```
:/opt/docker_containers/pi_hole $ sudo usermod -aG docker
```

You will need to log out and back in after the group change.

10. Once logged back in, navigate to where the .yaml file was created.

cd /opt/docker_containers/pi_hole/

```
:/opt/docker_containers $ cd /opt/docker_containers/pi_hole
:/opt/docker_containers/pi_hole $ ls
pi_hole_conf.yaml  volume
```

11. Now call the .yaml file to create the pihole container.

sudo docker compose -f pi_hole_conf.yaml up -d

```
:/opt/docker_containers/pi_hole $ sudo docker compose -f pi_hole_conf.yaml up -d
[+] up 14/16
 ⠸ Image pihole/pihole:latest [▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 43.85MB / 43.85MB Pulling
```

```
[+] up 18/18
 ✔ Image pihole/pihole:latest Pulled
 ✔ Network pi_hole_default    Created
 ✔ Container [redacted]       Started
:/opt/docker_containers/pi_hole $
```

Unbound Install and Configuration:

1. Before we install Unbound, lets ensure your network gear and or ISP are not blocking. We will install dnsutils to be able to use dig. Parts of bind9-dnsutils will be needed later in the KeepAliveD section.

   sudo apt install bind9-dnsutils -y

```
:/opt/docker_containers/pi_hole $ sudo apt install bind9-dnsutils -y
Building dependency tree... 94%
```

Run the cmds against your Pi,

UDP Check:

dig @198.41.0.4 . NS +norec +time=3

Your flags: section should look like so,

```
:/opt/docker_containers/pi_hole $ dig @198.41.0.4 . NS +norec +time=3

; <<>> DiG 9.20.29-1~deb13u1-Debian <<>> @198.41.0.4 . NS +norec +time=3
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 55216
;; flags: qr aa; QUERY: 1, ANSWER: 13, AUTHORITY: 0, ADDITIONAL: 27
```

If you see the following instead, you are being intercepted.

flags: qr ra; QUERY: 1, ANSWER: 13, AUTHORITY: 0, ADDITIONAL: 0

TCP Check:

dig @198.41.0.4 . NS +norec +tcp +time=3

Your flags: section should look like so,

```
:/opt/docker_containers/pi_hole $ dig @198.41.0.4 . NS +norec +tcp +time=3

; <<>> DiG 9.20.29-1~deb13u1-Debian <<>> @198.41.0.4 . NS +norec +tcp +time=3
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 54510
;; flags: qr aa; QUERY: 1, ANSWER: 13, AUTHORITY: 0, ADDITIONAL: 27
```

Again, if you see the following instead, you are being intercepted.

flags: qr ra; QUERY: 1, ANSWER: 13, AUTHORITY: 0, ADDITIONAL: 0

Root Hit Identity Check:

dig @198.41.0.4 version.bind CH TXT +time=3

Your version.bind. section should look like so,

```
:/opt/docker_containers/pi_hole $ dig @198.41.0.4 version.bind CH TXT +time=3

; <<>> DiG 9.20.29-1~deb13u1-Debian <<>> @198.41.0.4 version.bind CH TXT +time=3
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 5536
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;version.bind.                  CH      TXT

;; ANSWER SECTION:
version.bind.           0       CH      TXT     "ATLAS"
```

If you get anything other than ATLAS or return SERVFAIL your traffic is being intercepted.

2. If you passed the checks above, you are now ready to install Unbound! As a Pi does not have a CMOS battery, device time will need to be handled. If you are handling NTP somewhere, ensure the pi is managed by it. Else, you can install fake-hwclock to manage time by periodically writing time to disk so you don't break Unbound either Recursion or Forwarding when using Unbound, breaking all DNS for your network. fake-hwclock manages itself and just need to be installed.

   sudo apt install fake-hwclock -y

```
:/etc/unbound/unbound.conf.d $ sudo apt install fake-hwclock -y
```

3. Now install Unbound!

   sudo apt install unbound -y

```
:/opt/docker_containers/pi_hole $ sudo apt install unbound -y
```

4. Create the .conf file and name it to how you plan to run Unbound. Be sure to reference the GitHub KB. https://docs.pi-hole.net/guides/dns/unbound/

   quad9-dot

   sudo nano /etc/unbound/unbound.conf.d/quad9-dot.conf

```
:/etc/unbound/unbound.conf.d $ sudo nano /etc/unbound/unbound.conf.d/quad9-dot.conf
```

   Be sure to add ip-freebind: yes somehwere in your server: section to prevent a possible boot-time race condition where Unbound fails to start because it tries to bind to a local IP address that the operating system hasn't fully initialized yet.

1. Now check the .conf,

   sudo unbound-checkconf

```
:/etc/unbound/unbound.conf.d $ sudo unbound-checkconf
unbound-checkconf: no errors in /etc/unbound/unbound.conf
```

2. In conjunction w/ using freebind, we are going to tell the Unbound service that if 100 seconds of retries (20 attempts × 5 seconds), and the unbound services is still not started, stop trying. This will pair with a script we will use below that will transition a Pi from the primary Pi to a secondary during a service issue. Be sure to put the changes in the "drop in" section. This keeps the changes post update.

   sudo systemctl edit unbound

```
:~ $ sudo systemctl edit unbound
```

   [Unit]
   After=network-online.target
   Wants=network-online.target
   StartLimitIntervalSec=300
   StartLimitBurst=20

   [Service]
   RestartSec=5

```
  GNU nano 8.4                                    /etc/systemd/system/unbound.service.d/.#override.confb5d63e0d2cb2782a *
### Editing /etc/systemd/system/unbound.service.d/override.conf
### Anything between here and the comment below will become the contents of the drop-in file

[Unit]
After=network-online.target
Wants=network-online.target
StartLimitIntervalSec=300
StartLimitBurst=20

[Service]
RestartSec=5

### Edits below this comment will be discarded


### /usr/lib/systemd/system/unbound.service
# [Unit]
# Description=Unbound DNS server
# Documentation=man:unbound(8)
# After=network.target
# Before=nss-lookup.target
# Wants=nss-lookup.target
#
# [Service]
# Type=notify
# Restart=on-failure
# Environment=DAEMON_OPTS=
# EnvironmentFile=-/etc/default/unbound
# ExecStartPre=-/usr/libexec/unbound-helper chroot_setup
# ExecStartPre=-/usr/libexec/unbound-helper root_trust_anchor_update
# ExecStart=/usr/sbin/unbound -d -p $DAEMON_OPTS
# ExecStopPost=-/usr/libexec/unbound-helper chroot_teardown
# ExecReload=+/bin/kill -HUP $MAINPID
#
# [Install]
# WantedBy=multi-user.target
```

   When you save and exit the file, it should state it edited file ok.

```
Successfully installed edited file '/etc/systemd/system/unbound.service.d/override.conf'.
```

3. Now check the .conf,

   sudo unbound-checkconf

```
:/etc/unbound/unbound.conf.d $ sudo unbound-checkconf
unbound-checkconf: no errors in /etc/unbound/unbound.conf
```

4. Restart the service so the new .conf is utilized.

   sudo service unbound restart

```
:/etc/unbound/unbound.conf.d $ sudo service unbound restart
```

5. Now let's ensure Unbound is working as expected. Note these tests and flag returns are for a forwarding unbound setup going to 9s w/ DoT.

   Unbound is running, listening on 5335, and can resolve names at all.

   dig pi-hole.net @127.0.0.1 -p 5335

```
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
```

   Unbound rejects data with a deliberately broken DNSSEC signature.

   dig @127.0.0.1 -p 5335 sigfail.verteiltesysteme.net

```
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1
```

   That unbound validates a correctly signed domain.

   dig @127.0.0.1 -p 5335 sigok.verteiltesysteme.net

```
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1
```

   Forces unbound to contact its upstream, ie 9s. The 853 forced DoT.

   dig @127.0.0.1 -p 5335 test$RANDOM.example.com; sudo ss -tnp | grep 853

   The 149.112.112.112 and 9.9.9.9 via unbound shows unbound was leveraged in directly talking to the forwarded upstream over DoT just fine.

```
ESTAB 0      0      [redacted]:50124      9.9.9.9:853   users:(("unbound",pid=264778,fd=14))
```

6. If you are running RaspberryPi OS or Debian, ensure unbound-resolvconf.service is disabled. It will conflict per the Unbound documentation in the GitHub link above.

   Check if it is enabled or not,

   systemctl is-enabled unbound-resolvconf.service

```
[redacted]:/etc/unbound/unbound.conf.d $ systemctl is-enabled unbound-resolvconf.service
enabled
```

   if not, disable it,

   sudo systemctl disable --now unbound-resolvconf.service

```
[redacted]:/etc/unbound/unbound.conf.d $ sudo systemctl disable --now unbound-resolvconf.service
[sudo] password for [redacted]:
Removed '/etc/systemd/system/unbound.service.wants/unbound-resolvconf.service'.
```

   Check to ensure it is disabled.

```
[redacted]:/etc/unbound/unbound.conf.d $ systemctl is-enabled unbound-resolvconf.service
disabled
```

7. Now ensure,

   resolvconf_resolvers.conf is not in /etc/unbound/unbound.conf.d/ if so delete it.

   ls /etc/unbound/unbound.conf.d/

   Check if /etc/resolvconf.conf exsists, if so, delete it.

   grep -n unbound_conf /etc/resolvconf.conf

```
:/etc/unbound/unbound.conf.d $ ls /etc/unbound/unbound.conf.d/
grep -n unbound_conf /etc/resolvconf.conf
quad9-dot.conf  remote-control.conf  root-auto-trust-anchor-file.conf
grep: /etc/resolvconf.conf: No such file or directory
```

Configuring KeepAliveD:

1. Install KeepAliveD.

   sudo apt install keepalived -y

```
:/etc/unbound/unbound.conf.d $ sudo apt install keepalived -y
Building dependency tree... 66%
```

   Before we get started in the configuration, be sure to check/modify your DHCP reservations before and after making changes. Specifically the future second Pi we will be configuring as well as the VRRP virtual IP that KeepAliveD needs to HA Pis for a single IP on a Router. If your Router requires a MAC when reserving an IP, you might want to add a Local Administered MAC (NOT a manufacturer-assigned one) ie, 02:20:20:20:20:20. These MACs can be anything but just make sure the SECOND hex values use 2, 6, A, or E, ie 0A:10:20:30:40:50 BUT be sure to log this somewhere so you don't overlap MACs. This is a router requirement for me. It might not be for you and is not a requirement for KeepAliveD.

   Ensure you have already installed bind9-dnsutils will be needed later in the KeepAliveD script section in its .conf.

   apt list bind9-dnsutils

```
:/etc/unbound/unbound.conf.d $ apt list bind9-dnsutils
bind9-dnsutils/stable-security,now 1:9.20.29-1~deb13u1 arm64 [installed]
bind9-dnsutils/stable-security 1:9.20.29-1~deb13u1 armhf
```

2. Let's fine our primary ethernet name and take note of it.

   ifconfig -a

```
:/etc/unbound/unbound.conf.d $ ifconfig -a
br-7294591c44de: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.18.0.1  netmask [redacted]  broadcast [redacted]
        inet6 [redacted]  prefixlen 64  scopeid 0x20<link>
        ether [redacted]  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet [redacted]  netmask [redacted]  broadcast [redacted]
        ether [redacted]  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 11 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet [redacted]  netmask [redacted]  broadcast [redacted]
        ether [redacted]  txqueuelen 1000  (Ethernet)
        RX packets 215879  bytes 197775002 (188.6 MiB)
        RX errors 0  dropped 1  overruns 0  frame 0
        TX packets 38824  bytes 4693003 (4.4 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 25  bytes 3544 (3.4 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 25  bytes 3544 (3.4 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

vethe3d16ff: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet6 [redacted]  prefixlen 64  scopeid 0x20<link>
        ether [redacted]  txqueuelen 0  (Ethernet)
        RX packets 1337  bytes 1199419 (1.1 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1964  bytes 2119460 (2.0 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

wlan0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether [redacted]  txqueuelen 1000  (Ethernet)
        RX packets 32457  bytes 4772446 (4.5 MiB)
        RX errors 0  dropped 1  overruns 0  frame 0
        TX packets 4251  bytes 351001 (342.7 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

3. Now let's create and paste in our configurations for keelalived.conf. Reference the projects KB.
   https://www.keepalived.org/documentation/user-guide/configuration-synopsis/

   sudo mkdir /etc/keepalived/

```
/etc $ sudo mkdir /etc/keepalived/
```



keep_alive_
d_master

sudo nano /etc/keepalived/keepalived.conf

```
:/etc/unbound/unbound.conf.d $ sudo nano /etc/keepalived/keepalived.conf
```

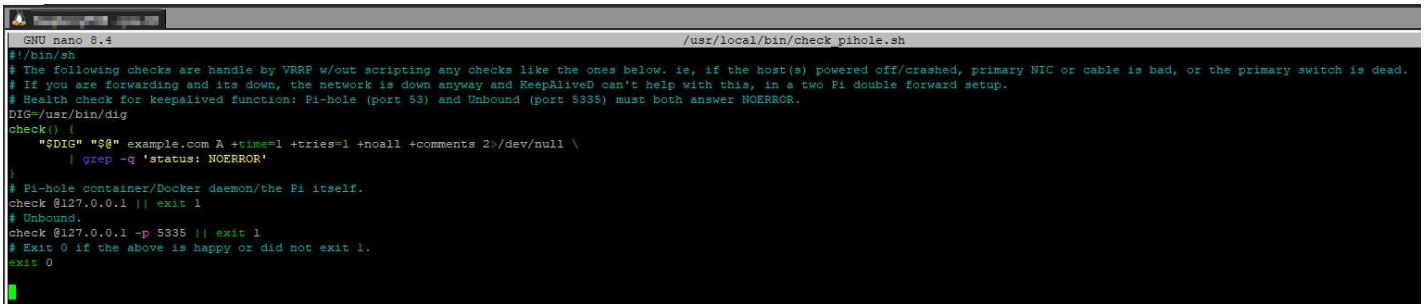
4. Create the script for keepalived and past in the code.

   sudo nano /usr/local/bin/check_pihole.sh



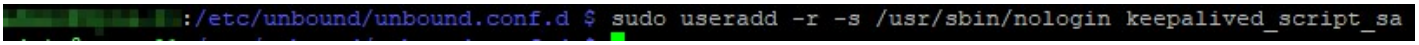
   check_piho
   le

```
#!/bin/sh
# The following checks are handle by VRRP w/out scripting any checks like the ones below. ie, if
the host(s) powered off/crashed, primary NIC or cable is bad, or the primary switch is dead.
# If you are forwarding and its down, the network is down anyway and KeepAliveD can't help with
this, in a two Pi double forward setup.
# Health check for keepalived function: Pi-hole (port 53) and Unbound (port 5335) must both
answer NOERROR.
DIG=/usr/bin/dig
check() {
   "$DIG" "$@" example.com A +time=1 +tries=1 +noall +comments 2>/dev/null \
      | grep -q 'status: NOERROR'
}
# Pi-hole container/Docker daemon/the Pi itself.
check @127.0.0.1 || exit 1
# Unbound.
check @127.0.0.1 -p 5335 || exit 1
# Exit 0 if the above is happy or did not exit 1.
exit 0
```



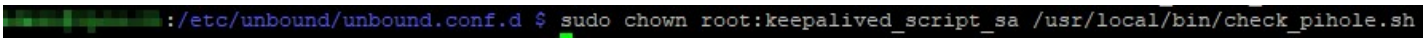
5. Create keepalived service account for vrrp_script block.

   sudo useradd -r -s /usr/sbin/nologin keepalived_script_sa



6. Change the group ownership of the script.

   sudo chown root:keepalived_script_sa /usr/local/bin/check_pihole.sh



7. 750 : 7 = root r/w/x 5 = group can r/x 0 = All other user are blocked.

   sudo chmod 750 /usr/local/bin/check_pihole.sh



8. Test the script we created.

   sudo keepalived -t -f /etc/keepalived/keepalived.conf

   You should get no output if there are no error found in the .conf.

```
:/etc/unbound/unbound.conf.d $ sudo keepalived -t -f /etc/keepalived/keepalived.conf
:/etc/unbound/unbound.conf.d $
```

9. Test script and permissions. Should print 0. If it says permission denied, the group or mode is wrong.

   sudo -u keepalived_script_sa /usr/local/bin/check_pihole.sh; echo $?

```
/etc/unbound/unbound.conf.d $ sudo -u keepalived_script_sa /usr/local/bin/check_pihole.sh; echo $?
0
```

10. Ensure KeepAliveD always starts.

    sudo systemctl enable keepalived

    Then reboot the pi and ensure the service is active.

    sudo reboot now

    systemctl is-active keepalived

```
:~ $ systemctl is-active keepalived
active
```

    We will test the HA after the second Pi is configured.

11. Check to ensure the VIP is active on the eth specified above.

    ip -br addr show eth0

    You should first see the Pis static IP + the VIP IP and the state is UP.

```
:~ $ ip -br addr show eth0
eth0             UP             /24 /24
:~ $
```

12. Let's check our script is working as expected and this Pi is set as the MASTER for KeepAliveD.

    systemctl status keepalived

    You might see some errors/issue as the services and script where coming up, but as long as the last script run returned 0, VRRP_Script sucseeded, and Entering MASTER STATE as the latest status for those checks and not the inverse you are good.

```
:~ $ systemctl status keepalived
● keepalived.service - Keepalive Daemon (LVS and VRRP)
     Loaded: loaded (/usr/lib/systemd/system/keepalived.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-10-11 17:26:46 EDT; 18h ago
 Invocation: 6b43293047964dd5a1c6e3f5557b0f58
       Docs: man:keepalived(8)
             man:keepalived.conf(5)
             man:genhash(1)
             https://keepalived.org
   Main PID: 1075 (keepalived)
      Tasks: 2 (limit: 806)
        CPU: 35min 17.751s
     CGroup: /system.slice/keepalived.service
             ├─1075 /usr/sbin/keepalived --dont-fork
             └─1081 /usr/sbin/keepalived --dont-fork

Oct 11 17:26:46 [redacted] Keepalived[1075]: NOTICE: setting config option max_auto_priority should result in better keepalived performance
Oct 11 17:26:46 [redacted] Keepalived[1075]: Starting VRRP child process, pid=1081
Oct 11 17:26:46 [redacted] Keepalived[1075]: Startup complete
Oct 11 17:26:46 [redacted] systemd[1]: Started keepalived.service - Keepalive Daemon (LVS and VRRP).
Oct 11 17:26:46 [redacted] Keepalived_vrrp[1081]: Script `check_pihole` now returning 1
Oct 11 17:26:46 [redacted] Keepalived_vrrp[1081]: VRRP_Script(check_pihole) failed (exited with status 1)
Oct 11 17:27:32 [redacted] Keepalived_vrrp[1081]: Script `check_pihole` now returning 0
Oct 11 17:27:37 [redacted] Keepalived_vrrp[1081]: VRRP_Script(check_pihole) succeeded
Oct 11 17:27:37 [redacted] Keepalived_vrrp[1081]: (VI_1) Entering BACKUP STATE
Oct 11 17:27:40 [redacted] Keepalived_vrrp[1081]: (VI_1) Entering MASTER STATE
:~ $
```

```
Oct 11 17:26:46 [redacted] systemd[1]: Started keepalived.service - Keepalive Daemon (LVS and VRRP).
Oct 11 17:26:46 [redacted] Keepalived_vrrp[1081]: Script `check_pihole` now returning 1
Oct 11 17:26:46 [redacted] Keepalived_vrrp[1081]: VRRP_Script(check_pihole) failed (exited with status 1)
Oct 11 17:27:32 [redacted] Keepalived_vrrp[1081]: Script `check_pihole` now returning 0
Oct 11 17:27:37 [redacted] Keepalived_vrrp[1081]: VRRP_Script(check_pihole) succeeded
Oct 11 17:27:37 [redacted] Keepalived_vrrp[1081]: (VI_1) Entering BACKUP STATE
Oct 11 17:27:40 [redacted] Keepalived_vrrp[1081]: (VI_1) Entering MASTER STATE
```

13. Now let's ensure our flow is still working over the VIP (client → VIP → Pi-hole → Unbound → Quad9 over DoT). On a different device, ideally the second Pi we will configure next w/ Dig already installed. Note X are your VIP.

    dig @XXX.XXX.XXX.XXX google.com

    You should see NOERROR and only one DNS server returning the request. In my case the 142.X.X.X is 9s. The blanked out sections should show your VIP.

```
[redacted]:~ $ dig @[redacted] google.com

; <<>> DiG 9.20.29-1~deb13u1-Debian <<>> @[redacted] google.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 44635
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;google.com.                    IN      A

;; ANSWER SECTION:
google.com.             300     IN      A       142.251.210.110

;; Query time: 43 msec
;; SERVER: [redacted]#53([redacted]) (UDP)
;; WHEN: Mon Oct 12 13:53:51 EDT 2026
;; MSG SIZE  rcvd: 55
```

    We can then ensure PiHole is blocking domains/ads. Note X are your VIP.

    dig @ XXX.XXX.XXX.XXX doubleclick.net

    If the answer came back as 0.0.0.0, its blocking as it should.

```
;; ANSWER SECTION:
doubleclick.net.        2       IN      A       0.0.0.0
```

    <u>Unbound and KeepAliveD Post Checks</u>

1. Now that the above has been configured, reboot the pi and check the Unbound services and KeepAliveD script.

   sudo reboot now

   When the pi is back up,

   sudo ss -lunpt | grep 5335

sudo -u keepalived_script_sa /usr/local/bin/check_pihole.sh; echo $?

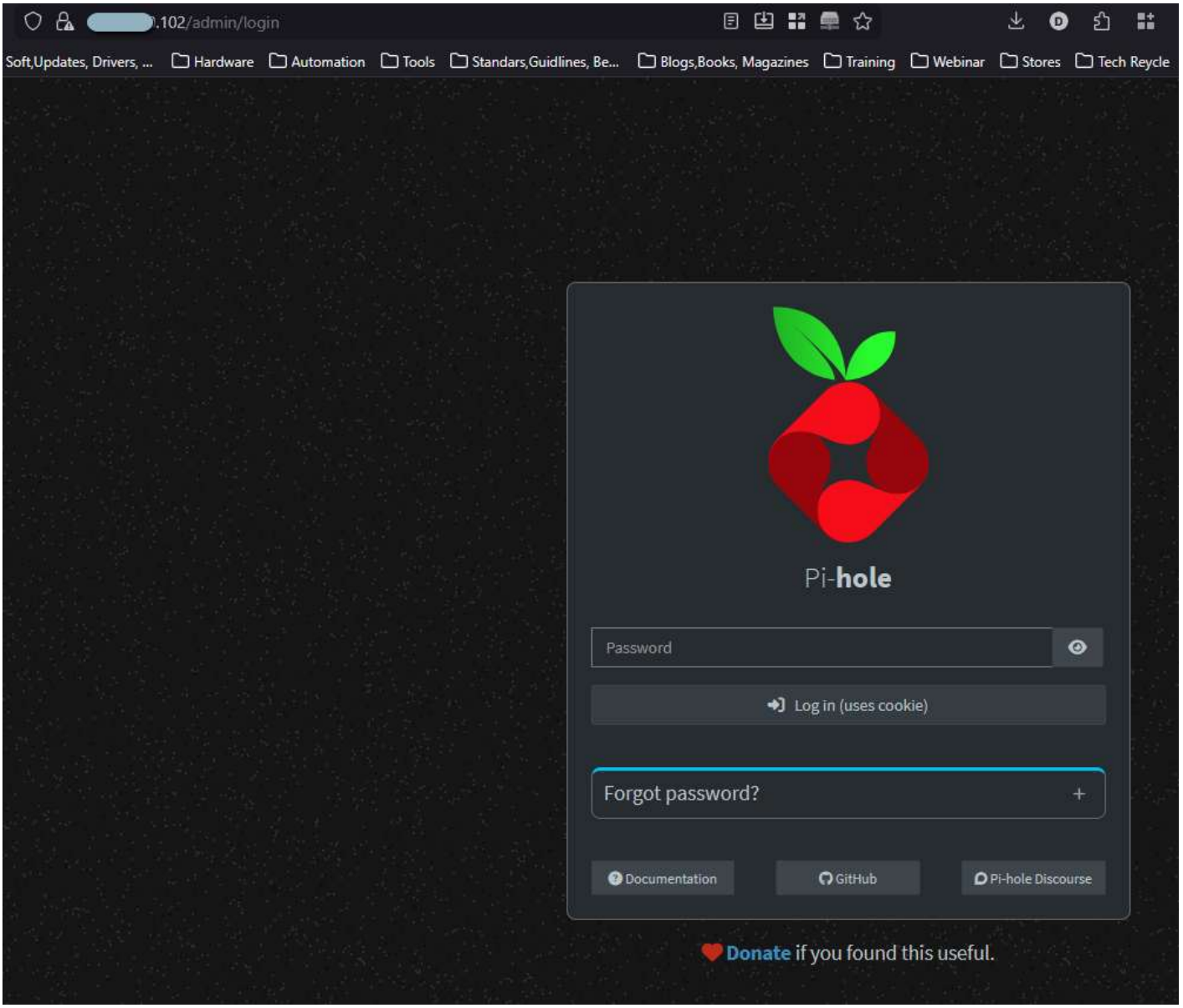
Ensure that the UDP checks all show unbound as the service and the script shows 0.

```
[redacted]:~ $ sudo ss -lunpt | grep 5335
[sudo] password for admin:
udp   UNCONN 0      0          [redacted]:5335       0.0.0.0:*    users:(("unbound",pid=1112,fd=5))
udp   UNCONN 0      0          127.0.0.1:5335        0.0.0.0:*    users:(("unbound",pid=1112,fd=3))
tcp   LISTEN 0      256        [redacted]:5335       0.0.0.0:*    users:(("unbound",pid=1112,fd=6))
tcp   LISTEN 0      256        127.0.0.1:5335        0.0.0.0:*    users:(("unbound",pid=1112,fd=4))
[redacted]:~ $ sudo -u keepalived_script_sa /usr/local/bin/check_pihole.sh; echo $?
0
[redacted]:~ $
```

Configuring PiHole:

2. Before you make any changes to your router. Statically set a device's DNS to the VIP to ensure nothing breaks immediately. If browsing and expected ad blocking seems to be working, revert the devices DNS and continue.

3. Now access your PiHole Container via the Pi's IP https://XX.XX.XX.XXX/admin/login



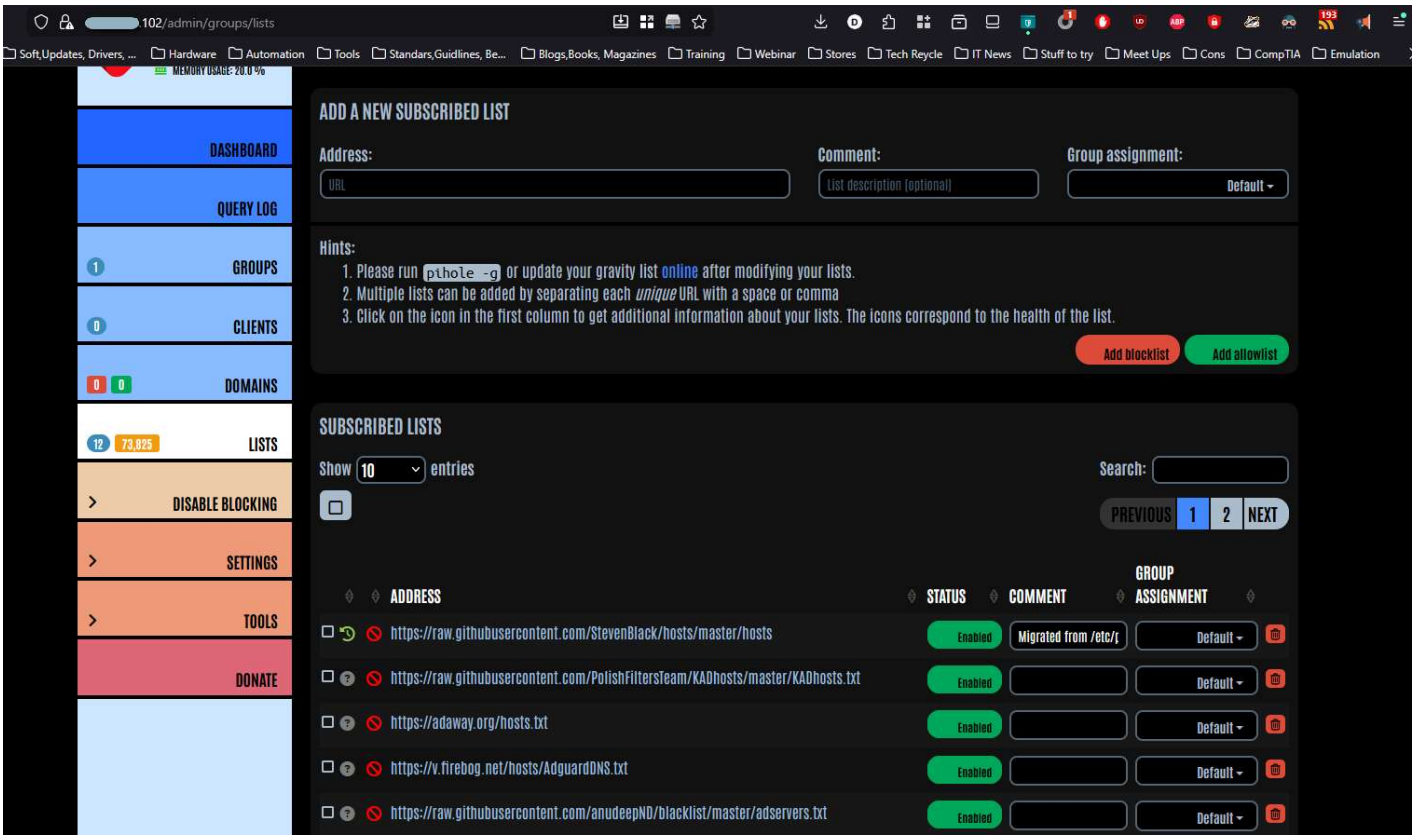
4. On your Router/Firewall put in the VIP as the primary DNS provider. No other DNS server can be specified or devices may use the second DNS server bypassing PiHole's DNS. Depending on your

DHCP setting it will take some time for the DNS change to reach hosts. You can speed it up via cli cmds if you wish. For Windows,

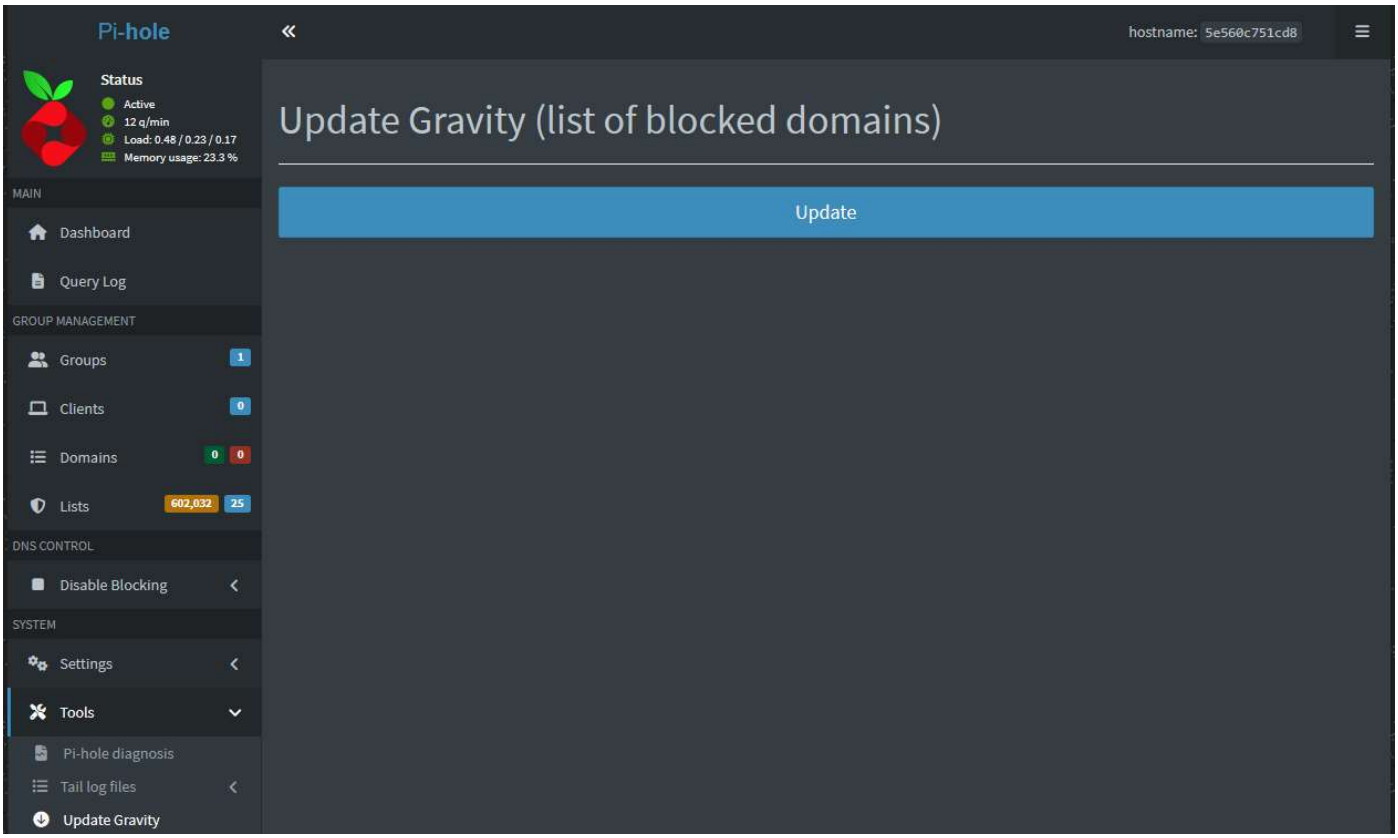
ipconfig /flush
ipconfig /release
ipconfig /renew

After some time you should start to see blocked requests in PiHole as well as other DNS data. It's not perfect but you can tune Pihole father, just be careful not to block needed content. Check out firebog.net for some ULR to add to the block list. Again less is more.

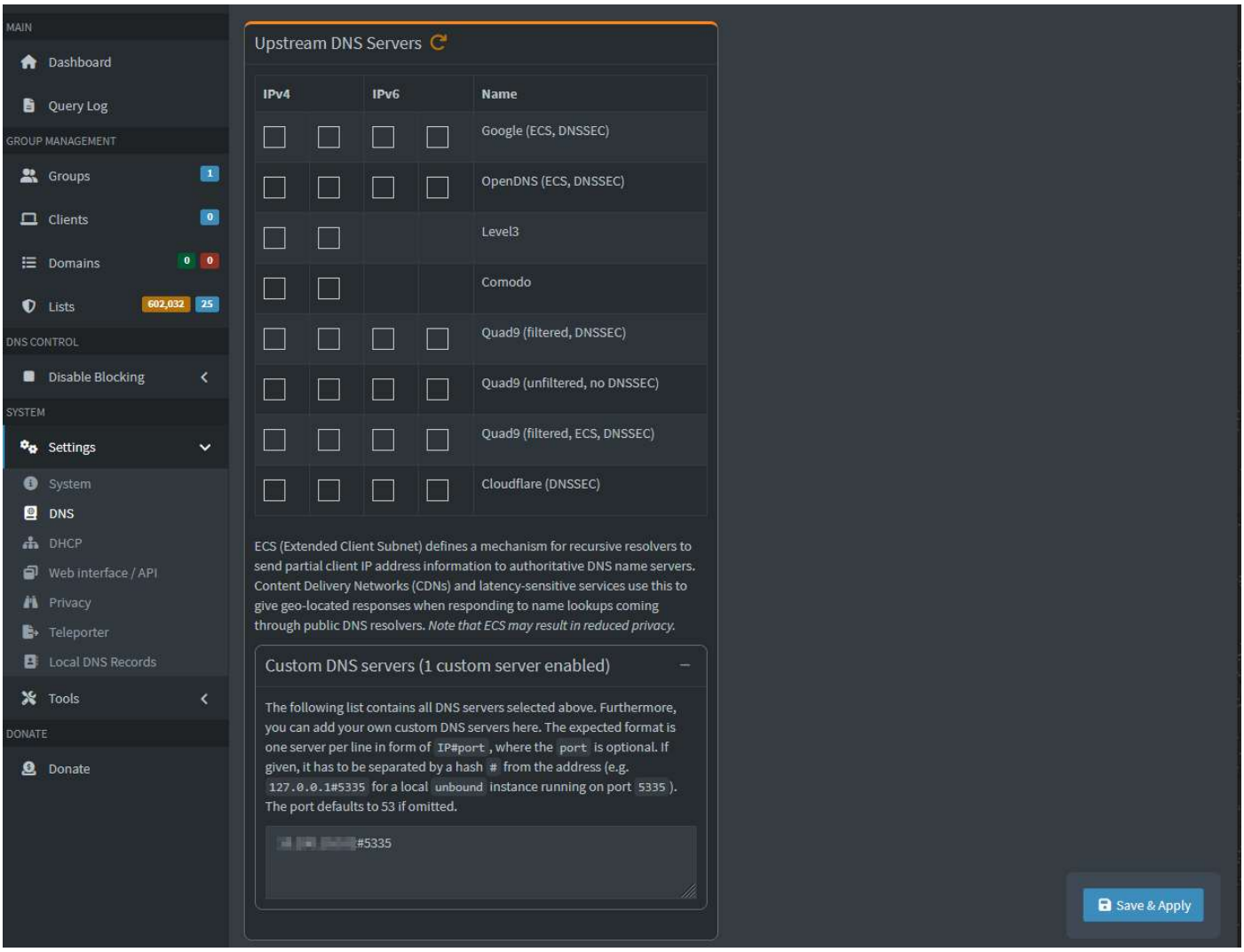
5. Additionally, there are block lists to add to pihole for beyond manually adding URLs from the FireBog.



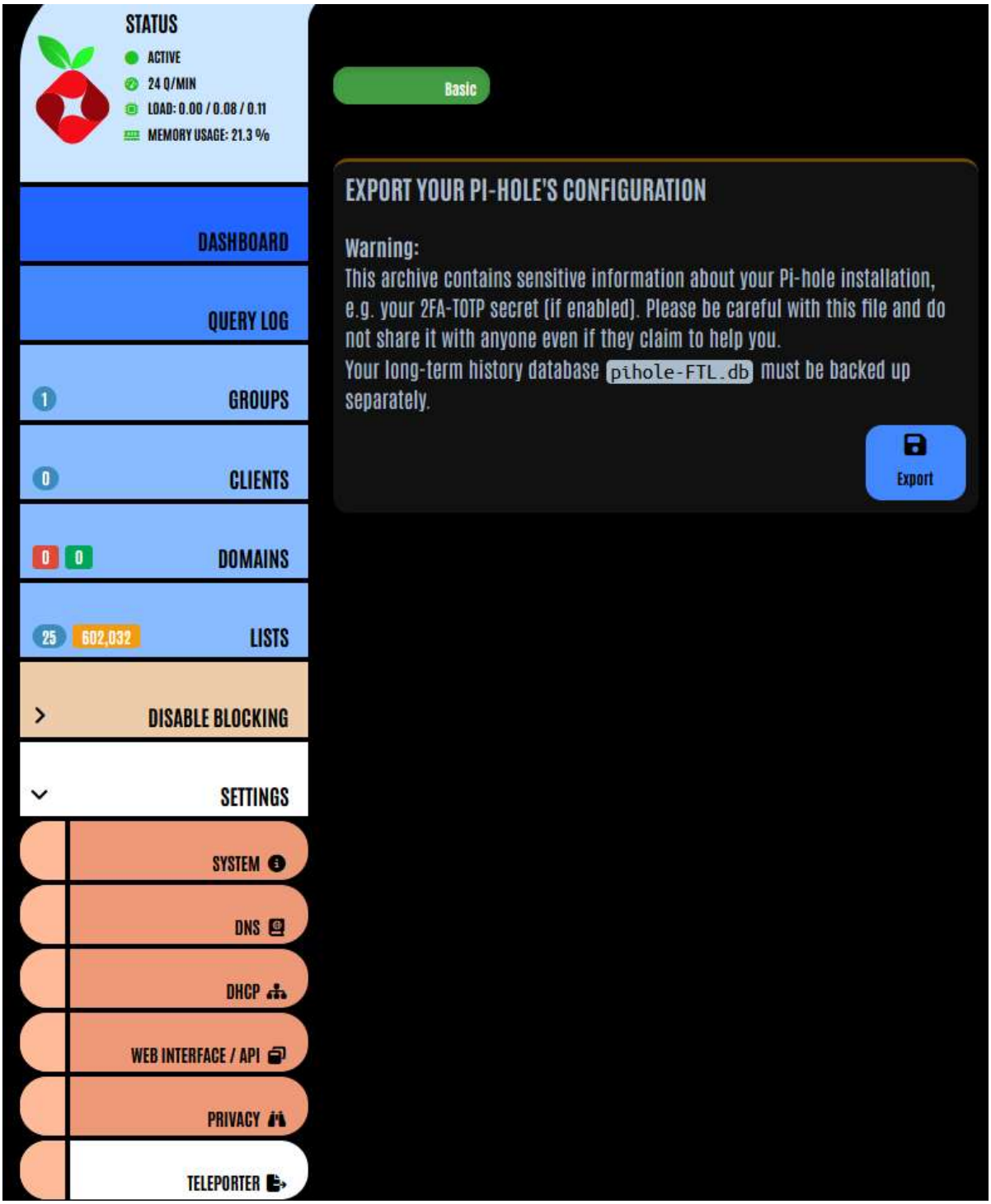
Don't forget to update gravity/sync the block lists you added and wait for it to state done.



6. Plug in just this Pis physical IP + Unbound port. DON'T put both IP + Unbound port.



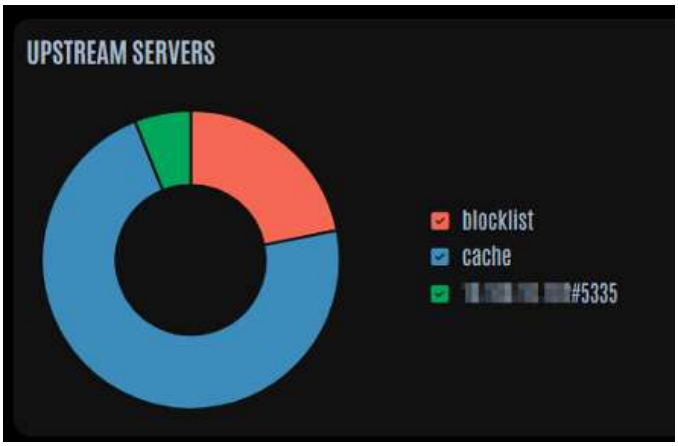
6. Once you feel you have your Pihole in a good spot, be sure to export the config as a backup.
   Settings > Teleporter > Export.





pi-hole_f4d
8d4b7c75...

As we are forwarding, eventually, you will only see your PiHoles IP, its cached lookups, and its blocks. google and or other previous providers will be cycled out eventually.



Links:

1. High Endurance Micro SD card - https://www.sdcard.org/press/thoughtleadership/understanding-the-difference-between-consumer-and-industrial-sd-memory-cards/

2. Raspberry Pi Imager - https://www.raspberrypi.com/software/

3. Statically set IP Shell script - https://github.com/XXLMandalorian013/Linux/blob/main/Scripts/Network/nmcli/ethernet/connect_ethernet_static.sh

4. Docker Install Shell script - https://github.com/XXLMandalorian013/Linux/blob/main/Scripts/Software/Installers/docker/64_bit/install_docker_64bit.sh

5. PiHole Install - pi-hole/docker-pi-hole: Pi-hole in a docker container

6. Unbound - https://docs.pi-hole.net/guides/dns/unbound/

7. KeepAliveD - https://www.keepalived.org/documentation/user-guide/configuration-synopsis/

8. Block lists/FireBog - https://firebog.net/